Plataformas de Desenvolvimento

24/25

Mestrado em Informática e Sistemas

1

REST API

- Why REST?
- RESTful (and Restful versus REST-like)
- REST best pratices

off-topic:

- API Management
- Security

2

Why REST

- 1991 - Common Object Request Broker Architecture (CORBA)
  - Supports multi language
- 1998 – Simple Objcet Access Protocol (SOAP)
  - XML exclusive
  - Well documented (WSDL)
  - E.g. https://ngrashia.medium.com/compiled-list-of-free-and-working-wsdl-urls-a7a54be6b91
  - http://www.dneonline.com/calculator.asmx?WSDL

- Both are complex
- Stiff contract between server and client
- Very dependent of the message bus

3

Why REST

- SOAP/XML is complex and network consuming
  - Mobile phones and Data connections
    - Passing XML -> network bandwidth
    - Memory consumption parsing XML
    - More battery consumed

- Normally REST is used with HTTP and Json

4

Why REST

- REpresentational State Transfer (REST)
  - defined by Fielding in his PhD thesis in 2000

- A design to fix earlier Remote Procedure Call (RPC) designs:
  - Simplicity in the contract between Client(s) and Server
  - Flexibility
  - Multiple data formats supported (XML, Json, CSV, …)
  - Set of design principals (RESTFul principals)
  - Independent of the bus channel and technology (e.g. HTTP)

5

Why REST

- Widely adopted by the industry to improve custom APPs:
  - Data Integration (e.g. FedEx)
  - Social (Facebook, Youtube, Twitter)
  - …

- Several API Payment Models
  - Developer pays:
    - Transaction (e.g. Paypal 2.9% by transaction)
    - Subscription (e.g. Salesforce $25/month for 10,000 daily calls)
    - Pay-as-you-go (e.g. Amazon Web Services)
  - Enterprise pays (Amazon associates - commission rate from 1 to 10 percent; Ebay).
  - Free (Facebook, Linkedin, …)
  - Freemium (e.g. :
    - Google Maps - initial credit plus Montly Volume Range cost;)
    - IBM Watson Assistant - Free with some features available

6

Why REST

- Large number of providers to manage REST API:
  - Service Level Aggreement (SLA)
  - Traffic Management
    - Quota (maximum per unit of time)
    - Rate limit (concurrent connections)
  - Security
    - OAUTH
    - Token management
    - Intrusion Detection Systems (IDS)
  - Analytics
  - Lifecycle
  - Monetization
- Apigee (Google Cloud Platform)
- IBM API Platform

7

RESTful



- Normally called RESTlike API or RESTish
  - May use Json
  - May use Resources
- To be RESTful MUST obey to REST API architectural constraints

8

RESTful

- Architectural constraints
  - Client-Server
  - Stateless
  - Cache
  - Uniform Interface
  - Layered System
  - Code-On-Demand (optional)

https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

9

RESTful - Client-Server

"Separation of **concerns** is the principle behind the client-server constraints. By **separating the user interface concerns from the data storage concerns**, we improve the **portability** of the user interface across multiple platforms and **improve scalability by simplifying the server components**. Perhaps most significant to the Web, however, is that the separation allows the components to **evolve independently**, thus supporting the Internet-scale requirement of multiple organizational domains."



Figure 5-2. Client-Server

https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

10

RESTful - Stateless

"communication must be stateless in nature, (…) such that each request from client to server **must** contain all of the information necessary to understand the request, and **cannot** take advantage of any stored context on the server. **Session** state is therefore kept entirely on the client."

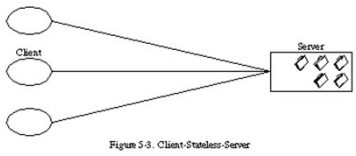


Figure 5-3. Client-Stateless-Server

https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

11

RESTful - Stateless

"This constraint induces the properties of visibility, reliability, and scalability.

(…)

The disadvantage is that it may decrease network performance by increasing the repetitive data (per-interaction overhead) sent in a series of requests, since that data cannot be left on the server in a shared context. In addition, (…) reduces the server's control over consistent application behavior,"

https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

12

RESTful - Cache

"To improve network efficiency, we add cache constraints (…). Cache constraints require that the data within a response to a request be implicitly or explicitly **labeled** as cacheable or non-cacheable. If a response is cacheable, then a client cache is given the right to reuse that response data for later, equivalent requests."

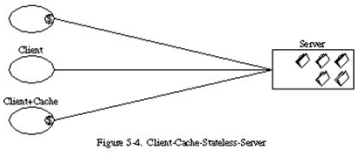


Figure 5-4. Client-Cache-Stateless-Server

https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

13

RESTful - Cache

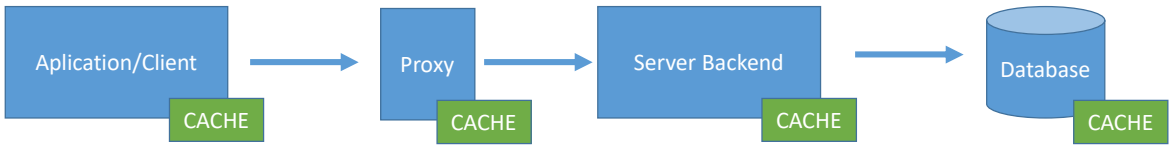
"advantage of adding cache constraints is that they have the potential to **partially or completely eliminate some interactions**, improving efficiency, scalability, and user-perceived performance by reducing the average latency of a series of interactions. The trade-off, however, is that a cache can decrease reliability if stale data within the cache differs significantly from the data that would have been obtained had the request been sent directly to the server."

https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

14

RESTful - Cache

- Multiple cache places



15

RESTful - Cache

- Server should specify **cache-control** headers to control caching behavior.

- Normally mark the **non**-cacheable responses.

  Although caching is an entirely OPTIONAL feature of HTTP, it can be assumed that **reusing a cached response is desirable** and that **such reuse is the default behavior** when no requirement or local configuration prevents it. Therefore, HTTP cache requirements are focused on preventing a cache from either storing a non-reusable response or reusing a stored response inappropriately, rather than mandating that caches always store and reuse particular responses.

  From https://datatracker.ietf.org/doc/html/rfc7234#page-5

16

RESTful - Cache

- Some cache-control mechanism:
  - No-Store – not allowed to be cached (sensitive data)
  - Private (cached only at User cache)
  - Public (Cached by any upstrem cache)
  - Max-age (Time in seconds for cache expiry)
  - S-maxage (Shared Cache expiry time)

The s-maxage directive does the same as max-age , but it is intended for caching proxies rather than for web clients.



17

RESTful - Cache

- Last-Modified – Last time resource was modified
- ETag – Unique Resource identifier



18

RESTful - Uniform Interface

"The central feature that distinguishes the REST architectural style from other network-based styles is its emphasis on a uniform interface between components.(…)

REST is defined by **four interface constraints**: identification of resources; manipulation of resources through representations; self-descriptive messages; and, hypermedia as the engine of application state"

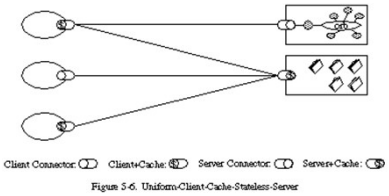


Figure 5-6. Uniform-Client-Cache-Stateless-Server

https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

19

RESTful - Uniform Interface

In practice the four interface constraints:

Identification of resources -> Resources are identified in the Request (URL/URI):

- GET http://api.example.com/store/**123**

Manipulation of resources -> Client receives and uses representation of the resource (URL/URI):

- DELETE http://api.example.com/store/**123**

URI - Uniform Resource Identifier

a unique sequence of characters that identifies a logical or physical resource

20

RESTful - Uniform Interface

In practice the four interface constraints:

Self-descriptive messages->

- Accepts: application/json
- Content-type
- HTTP Status Code

21

RESTful - Uniform Interface

In practice the four interface constraints:

hypermedia as the engine of application state -> HATEOAS

- Response data + **Actions available** for the resource

Hypermedia as the Engine of Application State (HATEOAS) is a constraint of the REST application architecture that distinguishes it from other network application architectures.

22

RESTful

Uniform Interface - HATEOAS

This excerpt from a sample response shows an array of HATEOAS links:

JSON

```
{
  "links": [{
    "href": "https://api-m.paypal.com/v1/payments/sale/36C38912MN9658832",
    "rel": "self",
    "method": "GET"
  }, {
    "href": "https://api-m.paypal.com/v1/payments/sale/36C38912MN9658832/refund",
    "rel": "refund",
    "method": "POST"
  }, {
    "href": "https://api-m.paypal.com/v1/payments/payment/PAY-
5YK922393D847794YKER7MUI",
    "rel": "parent_payment",
    "method": "GET"
  }]
}
```

Use the links in this example, as follows:

- To get more information about the request, combine the `GET` method and the target `href` of the `self` link.
- To request a refund, combine the `POST` method and the target `href` of the `refund` link.
- To get information about the parent payment, combine the `GET` method and the target `href` of the `parent_payment` link.

https://developer.paypal.com/docs/api/reference/api-responses/#hateoas-links

23

RESTful - Layered System

"(…)to further improve behavior for Internet-scale requirements(…) the layered system style allows an architecture to be composed of hierarchical layers by constraining component behavior such that each component **cannot "see"** beyond the immediate layer with which they are interacting (…) can also be used to improve system scalability by enabling **load balancing** of services across multiple networks and processors"

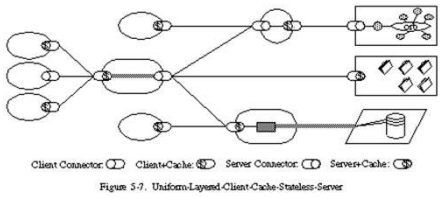


Figure 5-7. Uniform-Layered-Client-Cache-Stateless-Server

https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

24

RESTful - Layered System

- Reduces dependencies
- Architecture may evolve with the load (horizontal scalability)
- Layer changes impact ONLY one layer

https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

25

RESTful - Code-On-Demand

"REST allows client functionality to be extended by downloading and executing code in the form of applets or scripts. This simplifies clients by reducing the number of features required to be pre-implemented. Allowing features to be downloaded after deployment improves system extensibility. However, it also reduces visibility, and thus is only an **optional** constraint within REST."

e.g. of code-on-demand: Javascript

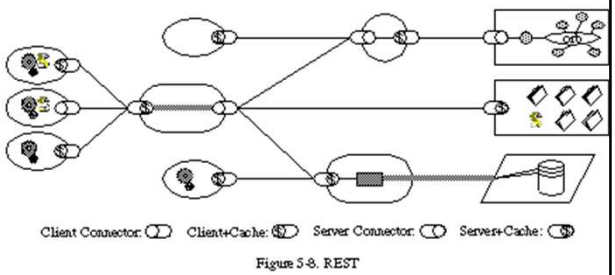


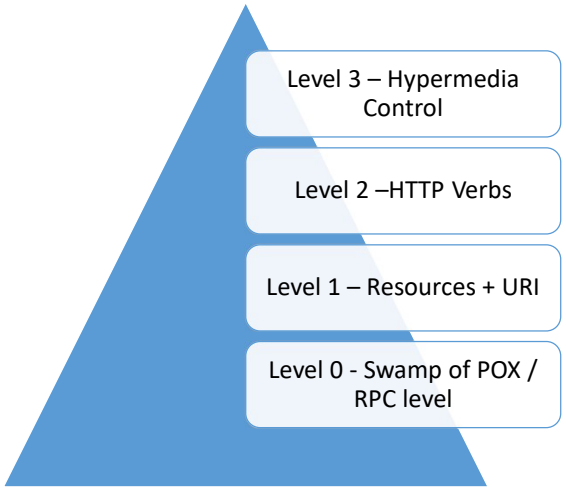
Figure 5-8. REST

https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

26

Richardson Maturity Model (RMM)

- Measure of Restfulness



Highest level of REST compliance

Lowest level of REST compliance

27

RESTful – Level 0 - Swamp of POX / RPC

POX – Plain Old XML

API uses XML as data format

It will use only one entry point (URI) and one kind of HTTP method

SOAP standard normally used

Don't know what are resources/URI (API endpoints focus on verbs)

- E.g. http://(...)/getCustomerData

28

RESTful – Level 1 – Resource + URI

API can distinguish between different resources.

uses multiple URIs to specific resources

E.g. distinguish between

http://example.org/article/1 and

http://example.org/article/2.

29

RESTful – Level 2 – HTTP Verbs

HTTP Verbs – GET, POST, PUT, DELETE, …

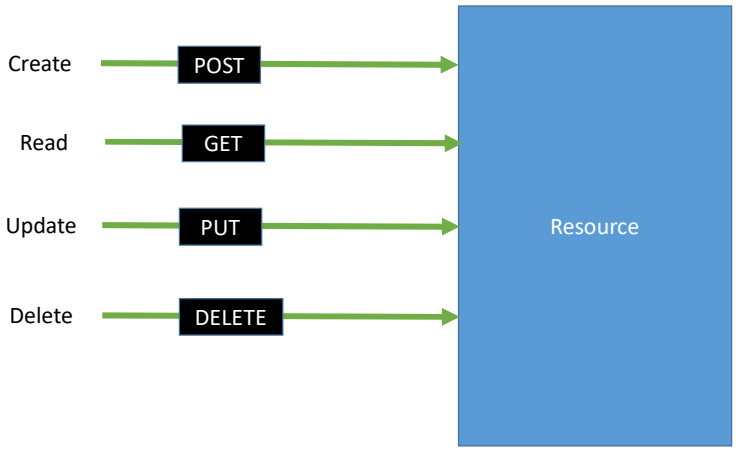
REST is completely protocol agnostic, so if you want to use a different protocol, your API can still be RESTful. Your API should use the **protocol properties (e.g. HTTP)**.

Manipulation MUST be made through the right HTTP Verb

CRUD Operations (Create Read Update Delete)

30

RESTful – Level 2 – HTTP Verbs



31

RESTful – Level 3 – Hypermedia control

The highest level, uses HATEOAS to deal with discovering the possibilities of your API towards the clients.

32

REST Best Practices – URL EndPoint

API URL Endpoint

http://domain/version/resource/{id}

33

REST Best Practices - Domain

API URL Endpoint

http://domain/version/resource/{id}

Domain and subdomains examples (normally without the WWW):

https://developer.uber.com/

https://api-m.paypal.com

34

REST Best Practices - Version

API URL Endpoint

http://domain/version/resource/{id}

Versioning allows to have multiple disruptive versions of the same API.

Isolates client upgrades from server upgrades

E.g. https://www.googleapis.com/youtube/v3

Note: It can be passed the version through HTTP Headers or Query Parameters

35

REST Best Practices - Resource

API URL Endpoint

http://domain/version/resource/{id}

Resource SHOULD be Nouns:

http://(...)/store

/item

/list

36

REST Best Practices - Resource

API URL Endpoint

http://domain/version/resource/{id}

Resource SHOULD be Nouns.

Operations for a Resource SHOULD be CRUD (Create Update Update Delete)

37

REST Best Practices - Resource

API URL Endpoint

http://domain/version/resource/{id}

In case the operation is not CRUD (search, calculations, compute, etc), being an **ACTION** it SHOULD be used a Verb (e.g. search).

Other option is to try to convert the **action** into a Noun operation

POST dogs/5/bark  POST dogs/5/command/

{

command: “bark”

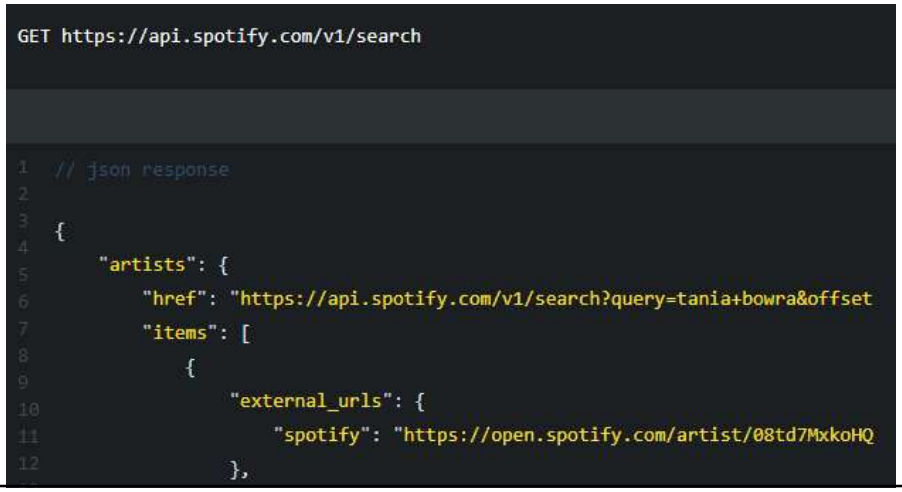
}

38

REST Best Practices – Resource (examples)

API URL Endpoint

http://domain/version/resource/{id}



```
GET https://api.spotify.com/v1/search
```

```
// json response

{
    "artists": {
        "href": "https://api.spotify.com/v1/search?query=tania+bowra&offset
        "items": [
            {
                "external_urls": {
                    "spotify": "https://open.spotify.com/artist/08td7MxkoHQ
                },
```

39

REST Best Practices – Resource (examples)

API URL Endpoint

http://domain/version/resource/{id}

GET /v1.2/estimates/price

https://developer.uber.com/docs/riders/references/api/v1.2/estimates-price-get

40

REST Best Practices - Resource

API URL Endpoint

http://domain/version/resource/{id}

Resource association to manipulate resource underneath other resource

E.g. Stores have Items

POST /store/{id-store}/item/{id-item}

E.g. People have friends

GET /person/{id-persion}/friends/{id-friend}

41

REST Best Practices - Resource

Resource operations:

PUT /dogs/5/toys/1	Change something of the toy 1 of dog 5
GET /dogs/5	Get ALL data from dog 5
POST /dogs/5/toys	Get dog 5 a new toy
DELETE /dogs/5/toys/2	Delete toy 2 from dog 5
GET /dogs?fur=brown	Get a list of all brown dogs

Rules:

GET operations are read only

POST creates; PUT creates or Updates

PUT/GET/DELETE are idempotent

42

REST Best Practices - Resource

Always use GET to retrieve information?

- If you have sensitive data
- If you have length constraints /URL normally limited to 2048 characters
- If you want to send complex structures (e.g. json)

You can use POST instead of GET (the opositive is not true)

43

REST Best Practices – HTTP Status Code

API URL Endpoint

http://domain/version/resource/{id}

3 digit that return STANDARD HTTP Return Codes

- Normally HTTP 200,201 and 202 for success.
- Don't create new error codes
- Don't have a wide range of error codes
- Don't use HTTP Status Code that no one understands the meaning:
  - E.g. HTTP Status: 418 I'm a teapot

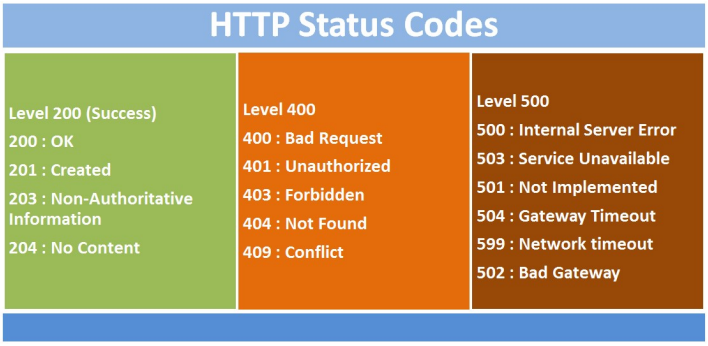
44

REST Best Practices – HTTP Status Code

API URL Endpoint

http://domain/version/resource/{id}

3 digit that return STANDARD HTTP Return Codes



45

REST Best Practices – HTTP Status Code

API URL Endpoint

http://domain/version/resource/{id}

3 digit that return STANDARD HTTP Return Codes

- DON'T use the HTTP STATUS CODE for Application Errors
- Use instead:
  - a custom HTTP response header: X-ERROR-CODE: 1525
  - OR a Json body response:

```
{
    "error": "auth-0001",
    "message": "Incorrect username and password",
    "detail": "Ensure that the username and password included in the request are correct",
    "help": "https://example.com/help/error/auth-0001"
}
```

46

REST Best Practices – Partial Responses

Different clients may need only a subset of responses (e.g. mobile phones for battery and network consumption).

Use Field projections (graphQL uses natively this approach):

If you want to get `id`, `bar`, and `baz` back in the response (but not `foo`, because it's irrelevant to your application), use a field projection:

https Copy

```
GET https://api.linkedin.com/v2/sampleService/42?fields=id,bar,baz
```

https://docs.microsoft.com/en-us/linkedin/shared/api-guide/concepts/projections?context=linkedin/context

47

REST Best Practices – Pagination

Different clients may need only a subset of the list (e.g. desktop versus mobile phones).

Offset pagination:

Sample Request

https Copy

```
GET https://api.linkedin.com/v2/{service}?start=10&count=10
```

Sample Response

JSON Copy

```
"elements": [
    {"Result #10"},
    {"Result #11"},
    {"Result #12"},
    {"Result #13"},
    {"Result #14"},
    {"Result #15"},
    {"Result #16"},
    {"Result #17"},
    {"Result #18"},
    {"Result #19"}
],
"paging": {
    "count": 10,
    "start": 10
}
```

https://docs.microsoft.com/en-us/linkedin/shared/api-guide/concepts/pagination?context=linkedin/context

48

REST Best Practices
Swagger/ OpenAPI Specification

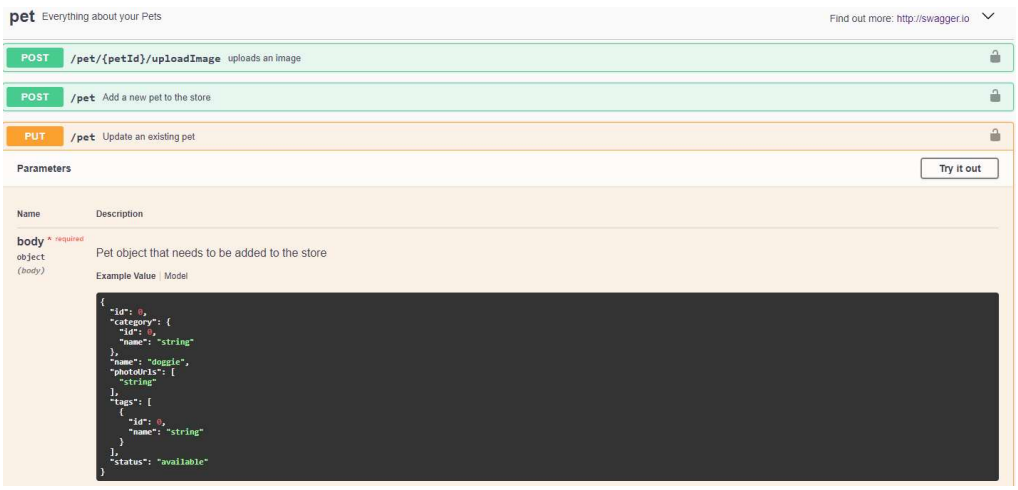
Document your API in a standard way

Generate the doc based on your code

Publish to QA testers/Client developers/FrontEnd developers

49

REST Best Practices
Swagger/ OpenAPI Specification



https://petstore.swagger.io/?_ga=2.43492773.1028391864.1620470099-1104341818.1618169128#/pet/updatePet

50

REST Implementation

Java - Jax-RS

```
@GET
@Path("{username}")
public Account getAccount(@PathParam("username") String username) {
    return accountRepository.findAccountByUsername(username);
}
```

Java - Spring-MVC

```
@RequestMapping(value="/{username}", method=GET)
public @ResponseBody Account getAccount(@PathVariable String username) {
    return accountRepository.findAccountByUsername(username);
}
```

51

REST Implementation

Python – Flask /Flask-restful

```
@app.route('/store/<string:name>/item' , methods=['POST'])
def create_item_in_store(name):
  request_data = request.get_json()
  for store in stores:
    if store['name'] == name:
        new_item = {
            'name': request_data['name'],
            'price': request_data['price']
        }
        store['items'].append(new_item)
        return jsonify(new_item)
  return jsonify ({'message' :'store not found'})
```

52

REST Implementation

Go(lang)

```
func (a *App) initializeRoutes() {
    a.Router.HandleFunc("/products", a.getProducts).Methods("GET")
    a.Router.HandleFunc("/product", a.createProduct).Methods("POST")
    a.Router.HandleFunc("/product/{id:[0-9]+}", a.getProduct).Methods("GET")
    a.Router.HandleFunc("/product/{id:[0-9]+}", a.updateProduct).Methods("PUT")
    a.Router.HandleFunc("/product/{id:[0-9]+}", a.deleteProduct).Methods("DELETE")
}
```

53

Some bibliography

- https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm
- https://developer.ticketmaster.com/products-and-docs/apis/getting-started/
- https://developer.twitter.com/en/products/twitter-api
- https://www.fedex.com/en-us/developer.html
- https://english.api.rakuten.net/dhlforyounl/api/dhlforyou-parcelshop-locator
- https://developers.google.com/youtube/v3

- https://www.business2community.com/affiliate-marketing/everything-you-need-to-know-about-amazons-affiliate-program-02246461
- https://www.g2.com/products/ibm-watson-assistant/pricing
- https://www.ibm.com/cloud/api-connect
- https://cloud.google.com/apigee
- https://en.wikipedia.org/wiki/Richardson_Maturity_Model
- https://www.slideshare.net/miredot/restful-api-design-and-documentation-an-introduction
- https://swagger.io/

54